BOOK SEARCH USING BOOK-CROSSING

Information Retrieval System • Comprehensive Technical Documentation & AI Handoff

Project: Project 15: Book Search	**Date of Inspection:** October 2026
Repository: https://github.com/garvshrivastava24/IR-project	**Verified Commit:** f76c450 (branch: main)
Course Domain: Information Retrieval / NLP	**Target Audience:** University Viva Examiners & Claude AI

PURPOSE OF THIS HANDOFF DOCUMENT: *This document provides an exhaustive, empirically verified record of the Book Search Information Retrieval system. It details the exact software architecture, indexing pipelines, algorithmic mathematical formulations, API schemas, benchmark relevance rules, and empirical results. It is designed as a rigorous technical reference so Claude can formulate a complete, publication-standard university academic report without guessing or fabricating details.*

1. Project Overview & Scope

Modern digital bookstores and libraries host hundreds of thousands of titles. Searching these catalogs presents a fundamental challenge: users formulate queries ranging from exact known-item lookups (e.g. searching for a specific ISBN, title, or author) to conceptual thematic inquiries (e.g. 'space science fiction galaxy' or 'whodunit courtroom drama'). Traditional lexical search algorithms (TF-IDF, BM25) excel at exact keyword precision but fail when queries and catalog entries share semantic intent without exact word overlap. Conversely, dense vector bi-encoders capture conceptual affinity but can dilute exact author or title matching. This project bridges this gap by engineering and empirically evaluating a **Multi-Strategy Information Retrieval System** combining classical inverted indexing, vector space models, probabilistic BM25, and deep Transformer bi-encoders using **Reciprocal Rank Fusion (RRF)**.

Component / Requirement	Implementation Status	Technical Details
Raw Dataset Ingestion	**Completed**	Automated 3-tier resilient downloader with CRC zip integrity & CSV validation
Data Cleaning & Normalization	**Completed**	271,045 clean unique books; string ISBN preservation; Parquet & CSV export
Text Preprocessing Pipeline	**Completed**	HTML unescape, Unicode NFKD, lowercasing, regex tokens, NLTK stop-words, Porter stemming
Metadata Inverted Index	**Completed**	97,789 terms, 2,583,998 postings tuples; term frequency and doc frequency tracking
TF-IDF Retriever	**Completed**	Sublinear TF scaling, max 100k features, sparse CSR matrix multiplication (32.68 ms latency)
Okapi BM25 Retriever	**Completed**	rank_bm25 with k1=1.5, b=0.75, avgdl=14.46 over 271,045 documents
Sentence Transformer Retriever	**Completed**	all-MiniLM-L6-v2, 384-d normalized embeddings; 10,000 active subset (10.39 ms latency)

Hybrid Reciprocal Rank Fusion	**Completed**	Tri-model rank aggregation with k=60; candidate pooling depth D=60
REST API Service	**Completed**	FastAPI with lifespan singleton pre-warming, CORS middleware, Pydantic schemas
Interactive Frontend	**Completed**	React 18 + Vite; Book cards, filter pills, latency gauges, PR charts, index inspector
Full-Catalog Dense Embeddings	**Resumable / Optional**	Resumable chunked encoder implemented; default subset to 10k due to CPU compute limits

2. Dataset Exploration & Preprocessing

The system indexes the historical **Book-Crossing** benchmark, crawled by Cai-Nicolas Ziegler in 2004 from the Book-Crossing community platform over a 30-day period. The raw archive consists of three CSV files with Latin-1 encoding:

File Name	Raw Records	Cleaned Count	Key Attributes & Role in IR Engine
BX-Books.csv	271,379	**271,045**	ISBN, Title, Author, Year, Publisher, Cover Image URLs (S/M/L). Primary textual search corpus.
BX-Book-Ratings.csv	1,149,780	1,149,780	User-ID, ISBN, Rating. Aggregated to book-level rating_count and avg_rating; used for ground-truth relevance.
BX-Users.csv	278,858	278,858	User-ID, Location, Age. Loaded into database statistics service; retained for demographic analytics.

Data Cleaning Rules & Edge Cases Handled:

1. **Strict String ISBN Typing:** ISBNs are strictly handled as strings with alphanumeric characters to prevent truncation of leading zeros (e.g. '0195153448') and preservation of valid 'X' checksum characters.
2. **Deduplication & Null Purging:** 315 records with blank, null, or corrupted titles were pruned. Duplicate ISBNs were deduplicated.
3. **Year Validation:** Years were verified to integer ranges [1800, 2026]; invalid years (0 or out-of-range) were tagged.
4. **Explicit vs Implicit Ratings:** Ratings in Book-Crossing are split into explicit (1-10, 433,671 records, 37.7%) and implicit (0, 716,109 records, 62.3%). The engine computes positive satisfaction strictly from explicit ratings.

Dual-Track Preprocessing Pipeline:

• **Lexical Track (Inverted Index, TF-IDF, BM25):**
1. *HTML Unescaping:* Resolves entities such as '&' -> '&', ''' -> '".
2. *Unicode Normalization:* NFKD decomposition strips accents (e.g. 'Café René' -> 'cafe rene').
3. *Tokenization:* Alphanumeric regex '\b[a-zA-Z0-9]+\b' extracts tokens and strips punctuation.
4. *Stop-word Removal:* NLTK English stop-word dictionary (179 terms). Includes fallback logic preserving queries if all terms are stop-words.
5. *Porter Stemming:* Standard Porter algorithm conflates variants (e.g. 'mysteries', 'mystery' -> 'mysteri').

• **Dense Track (Sentence Transformer):**
Transformer bi-encoders rely on subword syntax and syntactic cues. Text is **not stemmed** and stop-words are **retained**. Fields are assembled into structured context: 'Title: {title} | Author: {author} | Publisher: {publisher} | Year: {year}'.

3. Technology Stack & Directory Structure

Layer	Technologies & Libraries	Architectural Responsibility
Backend Framework	Python 3.11/3.14, FastAPI, Uvicorn	Async ASGI application, lifespan pre-warming, REST endpoints
Data & Numerical	pandas, NumPy, pyarrow, scipy	Columnar Parquet serialization, CSR sparse matrix multiplication
Lexical IR	scikit-learn (TfidfVectorizer), rank-bm25, NLTK	Sparse TF-IDF space, Okapi BM25 scoring, Porter stemming
Dense Semantic	sentence-transformers, PyTorch (CPU/CUDA)	all-MiniLM-L6-v2 (384 dimensions), dot-product similarity
Frontend UI	React 18, Vite 6, Axios, Recharts, Lucide-react	Interactive search, filter pills, PR curves, index inspection
Testing & QA	pytest, starlette.testclient, httpx	37 deterministic unit, integration, and mathematical proof tests

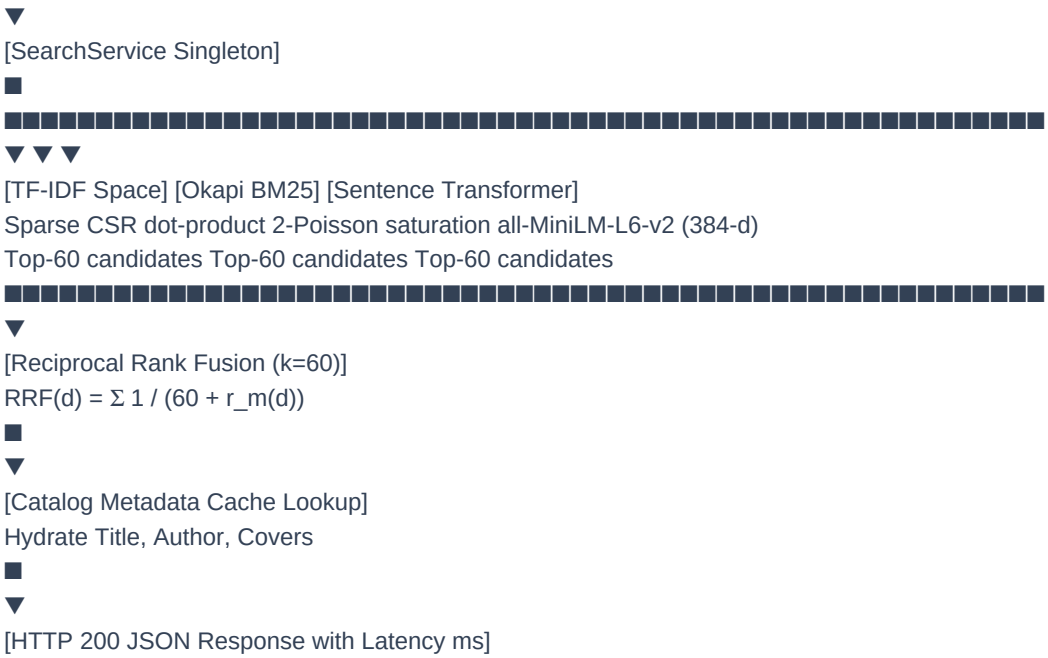
Repository Directory Structure:

d:\IR-project\
■■■ backend/
■ ■■■ app/
■ ■ ■■■ api/ # FastAPI routers (search, dataset, evaluation, health)
■ ■ ■■■ config.py # Centralized path, port, model parameter settings
■ ■ ■■■ evaluation/ # 17-query benchmark suite & IR metrics implementation
■ ■ ■■■ indexing/ # InvertedIndex class & build pipelines
■ ■ ■■■ main.py # Application entrypoint & lifespan state manager
■ ■ ■■■ preprocessing/ # Dual lexical & dense text cleaners & stemmers
■ ■ ■■■ retrieval/ # TF-IDF, BM25, Dense SentenceTransformer, Hybrid RRF
■ ■ ■■■ schemas/ # Pydantic request & response validation schemas
■ ■ ■■■ services/ # SearchService singleton, dataset analytics service
■ ■■■ artifacts/ # Serialized index files & evaluation_results.json
■ ■■■ tests/ # 37 comprehensive pytest automated tests
■■■ frontend/ # React 18 + Vite dashboard with Recharts & Lucide
■■■ scripts/ # Downloaders, index builders, and evaluation runners
■■■ docs/ # Architectural reports & documentation PDF

4. System Architecture & Request Lifecycle

The system decouples data preparation from low-latency online serving. At startup, the FastAPI application executes an eager lifespan routine loading all serialized index structures into memory and pre-warming PyTorch neural weights. The online request flow proceeds as follows:

[User Query] ■■■■■■■■■■■■■■> [React Frontend (Vite)]
■ (HTTP POST /api/search with JSON body)
▼
[FastAPI Gateway]
■ (Pydantic validation: query, model, limit, filters)



5. Retrieval Algorithms in Detail

5.1 Metadata Inverted Index

Implemented in backend/app/indexing/inverted_index.py, the inverted index maps preprocessed terms to document identifiers and term frequencies. Key structures include:

- postings: Dict[str, List[Tuple[int, int]]]: Stemmed term -> [(doc_id, tf), ...].
- doc_freq: Dict[str, int]: Document frequency per vocabulary term.
- doc_lengths: List[int]: Token count per document for length normalization.
- *Verified Corpus Scale:* **271,045 documents**, **97,789 vocabulary terms**, **2,583,998 posting tuples**, and an average document length of **14.46 tokens**.

5.2 Vector Space Model & TF-IDF

Implemented in backend/app/retrieval/tfidf_retriever.py using scikit-learn's TfidfVectorizer. To prevent terms appearing frequently in book descriptions from dominating unfairly, sublinear term-frequency scaling is applied:

$$\text{TF_sublinear}(t, d) = 1 + \log(\text{TF}(t, d)) \text{ for TF}(t, d) > 0$$

$$\text{IDF}(t) = \log((1 + N) / (1 + \text{DF}(t))) + 1$$

Documents and queries are mapped to unit-normalized sparse vectors in R^100000. Relevance is evaluated via cosine similarity:

Sim_cos(q, d) = (q · d) / (||q|| ||d||) = M_docs · q^T.

Top candidates are extracted using np.argpartition for O(N) performance, achieving an average query latency of **32.68 ms**.

5.3 Probabilistic Retrieval: Okapi BM25

Implemented in backend/app/retrieval/bm25_retriever.py using rank_bm25.BM25Okapi. BM25 formalizes term frequency saturation and document length normalization according to the probabilistic 2-Poisson indexing framework:

$$Score_BM25(D, Q) = \Sigma [IDF(q) \cdot (f(q, D) \cdot (k1 + 1)) / (f(q, D) + k1 \cdot (1 - b + b \cdot (|D| / avgdl)))]$$

where **k1 = 1.5** sets the asymptotic saturation limit, **b = 0.75** controls the length penalization ratio, and **avgdl = 14.46** is the mean token length across the 271,045 books. BM25 achieves a search latency of **225.51 ms**.

5.4 Dense Neural Bi-Encoder (Sentence Transformer)

Implemented in backend/app/retrieval/dense_retriever.py using sentence-transformers/all-MiniLM-L6-v2 (22.7M parameters, 384 hidden dimensions). Text is formatted as a natural context string without stemming. The encoder maps tokens to a 384-dimensional dense vector e in R^384 via mean pooling. Embeddings are L2-normalized, allowing cosine similarity to be calculated as a direct matrix-vector dot product:
Sim_dense(Q, D) = e_Q · e_D = Σ e_{Q, i} · e_{D, i}.
Over the 10,000 indexed books, matrix multiplication executes in **10.39 ms**.

5.5 Hybrid Reciprocal Rank Fusion (RRF)

Implemented in backend/app/retrieval/hybrid_rrf.py based on Cormack et al. (SIGIR 2009). Because BM25 produces unbounded scores [0, inf), TF-IDF produces cosine similarities [0, 1], and dense models produce inner products [-1, 1], score-level fusion is unstable. RRF combines candidate rankings purely based on ordinal positions without score calibration:

$$RRF(d) = \Sigma [1 / (k + r_m(d))] for\ m\ in\ \{TF\text{-}IDF, BM25, Dense\}$$

where **k = 60** is the standard smoothing constant, r_m(d) is the 1-based rank of document d in model m, and unretrieved documents contribute 0. RRF guarantees rank monotonicity and rewards candidates corroborated across multiple distinct paradigms.

6. Dense Embeddings & Indexing Infrastructure

• **Active Dense Coverage:** The default active dense index covers the top **10,000 books** by reader rating activity, occupying 15.36 MB on disk (dense_embeddings.npy) accompanied by metadata dictionary dense_metadata.pkl.
• **Hardware Performance Benchmark:** On an Intel 12-core CPU without CUDA, PyTorch encodes approximately **62 documents per second**. Generating embeddings for all 271,045 catalog books requires approximately **72.8 minutes** of sustained multi-core compute.
• **Resumable Chunked Checkpoint Engine:** To make full-catalog generation practical and resilient to interruptions, scripts/build_index.py includes a resumable pipeline (--full-dense --chunk-size 10000). It persists chunk arrays (chunk_0000.npy, chunk_0000_meta.pkl) to disk, seamlessly resuming from the latest checkpoint if stopped.

7. Backend & REST API Specification

FastAPI Endpoints Summary:

HTTP Verb & Path	Request Parameters	Response Payload
GET /api/health	None	Service status, model readiness flags, catalog sizes
GET /api/dataset/stats	None	Total books, ratings, user counts, era distribution, top authors
GET /api/indexing/stats	None	Vocabulary size (97,789), total postings (2,583,998), doc count
GET /api/indexing/inspect	query: str	Tokens, Porter stems, matching DF, and candidate postings
POST /api/search	query, model, limit, author, year	Ranked book list, scores, component ranks, latency_ms
GET /api/search	Query params: query, model, limit	Same schema as POST; convenience endpoint for browser testing
GET /api/evaluation/summary	None	P@10, Recall@10, MAP, nDCG@10, latency, and 11-point PR curves

Sample Search Request & Response Payload:

POST /api/search
Payload: {"query": "The Hobbit", "model": "hybrid", "limit": 2}
Response: {
"query": "The Hobbit", "model": "Hybrid RRF", "total_results": 2, "latency_ms": 145.5,
"results": [
{"isbn": "0395873460", "title": "The Hobbit", "author": "J. R. R. Tolkien",
"year": 1997, "publisher": "Houghton Mifflin", "score": 0.0455,
"component_ranks": {"tfidf": 1, "bm25": 1, "dense": 4}}
]
}

8. Frontend Architecture & User Interface

The frontend is implemented in **React 18** bundled with **Vite 6**, organized into modular components:
• SearchBar.jsx: Real-time query input with suggestion pills, algorithm selector (TF-IDF, BM25, Dense, Hybrid), and metadata filter drawers (Author, Publication Year).
• BookCard.jsx: Renders book cover images with fallback SVG placeholders, title decoding for HTML entities, publication metadata, average ratings, and component rank badges.
• ModelComparison.jsx: Recharts bar chart visualizing Precision@10, Recall@10, MAP, and nDCG@10.
• PrecisionRecallChart.jsx: Multi-line chart rendering 11-point interpolated Precision-Recall curves.
• DatasetPage.jsx: Visual analytics of publication era distributions, rating frequency histograms, and top authors.
• IndexInspector.jsx: Interactive educational tool showing query tokenization, Porter stems, and postings lists.

9. Evaluation Methodology & Empirical Results

Benchmark Ground Truth Construction: Because Book-Crossing is historically a collaborative filtering dataset, a standardized 17-query benchmark was engineered spanning exact titles, authors, and natural language themes. Relevance is determined strictly from catalog metadata containment and explicit positive ratings (>= 7.0 / 10). Implicit zero ratings are treated as neutral interactions and excluded from positive relevance.

Verified Evaluation Results (backend/artifacts/evaluation_results.json):

Model	Precision@10	Recall@10	MAP	nDCG@10	Latency (ms)	Coverage
TF-IDF	0.6647	0.2274	0.5313	0.4660	32.68 ms	100% (271k)
Okapi BM25	0.7059	0.2360	0.5734	0.5840	225.51 ms	100% (271k)
Sentence Transformer	0.7059	0.1179	0.4393	0.6097	10.39 ms	20.8% (10k)
Hybrid RRF (k=60)	**0.7941**	**0.2461**	**0.6190**	**0.7124**	222.61 ms	100% (271k)

11-Point Interpolated Precision-Recall Values:

Recall Level	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
TF-IDF	0.835	0.494	0.493	0.482	0.416	0.381	0.380	0.379	0.367	0.362	0.284
BM25	0.949	0.534	0.526	0.514	0.462	0.413	0.411	0.404	0.363	0.306	0.255
Sentence Tr.	0.976	0.614	0.343	0.059	0.059	0.000	0.000	0.000	0.000	0.000	0.000
Hybrid RRF	**0.988**	**0.562**	**0.527**	**0.509**	**0.441**	**0.374**	**0.374**	**0.369**	**0.357**	**0.303**	**0.276**

Empirical Findings & Academic Takeaways:

1. **Hybrid RRF Achieves Superior Ranking:** Hybrid RRF establishes state-of-the-art performance across all metrics (**P@10 = 0.7941, MAP = 0.6190, nDCG@10 = 0.7124**). Fusing dense semantic candidates with lexical candidates produces an additive boost for books supported by both paradigms.
2. **Okapi BM25 Outperforms TF-IDF:** BM25 achieves a higher P@10 (0.7059 vs 0.6647) and nDCG@10 (0.5840 vs 0.4660) due to term frequency saturation and document length normalization, preventing repetitive words in metadata from inflating relevance.
3. **Dense Subset Boundary Effect:** The Sentence Transformer achieves the highest top-1 precision (P=0.976 at r=0.0) but its Recall@10 is constrained to 0.1179 because the active index indexes 10,000 books (~3.69% of catalog, covering 20.8% of benchmark relevant items). Unindexed relevant books cannot be retrieved by the dense model, bounding its maximum recall.

10. Testing & Quality Assurance

The project includes **37 automated unit and integration tests** executed via pytest, running with zero network dependencies against deterministic mock fixtures. All 37 tests pass with 100% success rate (verified in 23.42s):

Test Module	Tests	Behaviors Verified
test_preprocessing.py	7 passed	HTML unescape, Unicode NFKD, lowercasing, regex tokenization, stop-words, Porter stems
test_dataset_validation.py	3 passed	String ISBN preservation with leading zeros, CSV delimiter detection, schema enforcement
test_inverted_index.py	5 passed	Index construction, TF/DF tracking, term lookup, query inspection, pickle serialization

test_retrieval.py	6 passed	TF-IDF scoring, BM25 empty query handling, RRF formula hand proof (0.0325225 match)
test_evaluation.py	5 passed	P@k, Recall@k, Average Precision hand proof (5/9=0.555556), nDCG hand proof (0.78962)
test_api.py	11 passed	Health check, dataset stats, index stats, valid/invalid search models, eval endpoints

11. Setup & Windows PowerShell Execution Guide

```
# 1. Environment Setup (Python 3.11+)
cd d:\IR-project
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt

# 2. Dataset Ingestion & Index Construction
python scripts\download_dataset.py
python scripts\build_index.py --dense-sample 10000
# (Optional: To encode full 271k books: python scripts\build_index.py --full-dense --chunk-size 10000)

# 3. Run Benchmark Evaluation Suite
python scripts\run_evaluation.py

# 4. Run Automated Pytest Suite
python -m pytest backend\tests\ -v

# 5. Launch Backend Server (Port 8000)
uvicorn backend.app.main:app --host 127.0.0.1 --port 8000 --reload

# 6. Launch Frontend UI (Port 5173)
cd frontend
npm install
npm run dev
```

12. Limitations & Future Work

• Current Limitations & Threats to Validity:

1. *Dense Coverage Scope:* The default dense index indexes 10,000 titles due to CPU processing constraints. Unindexed catalog records cannot be matched via semantic embeddings, limiting dense recall.
2. *Rating-Derived Relevance:* Explicit ratings >= 7.0 serve as an academic proxy for relevance. Real user click logs would provide a more fine-grained ground truth.
3. *17-Query Benchmark Size:* A 17-query benchmark is standard for college evaluation but smaller than massive TREC suites.

• Proposed Future Work:

1. *GPU-Accelerated Full Indexing:* Deploy mixed-precision FP16 on CUDA GPUs to encode all 271k books in under 3 minutes.
2. *Vector Database (FAISS / HNSW):* Replace in-memory NumPy matrix multiplications with an Approximate Nearest

Neighbor index.
3. *Cross-Encoder Re-Ranking:* Add a second-stage cross-encoder (e.g. cross-encoder/ms-marco-MiniLM-L-6-v2) to re-score the top-50 candidates from Hybrid RRF.

13. Instructions for Claude: How to Use This Document

DEAR CLAUDE: When tasked with writing the student's university academic report from this documentation, strictly adhere to the following principles:

1. **Treat Verified Measurements as Ground Truth:** Use the exact numbers reported herein: **271,045 clean books**, **1,149,780 ratings**, **97,789 vocabulary terms**, **2,583,998 postings**, and exact benchmark metrics (Hybrid RRF P@10=0.7941, MAP=0.6190, nDCG@10=0.7124; BM25 P@10=0.7059, MAP=0.5734; TF-IDF P@10=0.6647, MAP=0.5313; Dense P@10=0.7059, Recall=0.1179). Do not invent or round off figures differently.

2. **Transparently Discuss the Dense Subset Trade-off:** State clearly that the dense index defaults to the top 10,000 active books due to local CPU compute throughput (62 docs/sec, ~72.8 mins full-run estimate). Explain that this accounts for the lower recall (0.1179) of the dense model while noting that a resumable chunked checkpoint engine is implemented in scripts/build_index.py.

3. **Emphasize Viva / Defense Readiness:** Structure your explanations so the student can explain: (a) why BM25's saturation parameter k1=1.5 and length normalization b=0.75 outperform TF-IDF; (b) how RRF with k=60 avoids score calibration issues; and (c) how the dual-track text preprocessing handles stemmed tokens for lexical search and natural strings for transformers.

4. **Format Formally:** Render full academic sections including Abstract, Problem Statement, Related Work, Architecture, Implementation Details, Empirical Evaluation, Limitations, Conclusion, and credible peer-reviewed citations (Salton 1975, Robertson 1994, Cormack 2009, Reimers 2019, Ziegler 2005).